

專案開發紀錄：個人伺服器架構（Homelab）

背景資訊

在開發 **yt.py** 自動下載 YouTube 播放清單之後，我需要一個能在所有裝置上串流播放這些音樂的方案，於是架起了 **Navidrome** 音樂伺服器，這也是我第一次接觸 **Docker** 容器化部署。自架一個容器後再也停不下自架的樂趣。**Immich** 讓我不用再把所有照片交給 **Google Photos** 管理，**ConvertX** 讓我不用等線上轉檔網站的廣告跑完就能批次處理檔案，**AdGuard Home** 讓全家的裝置都不再看到廣告。一項一項疊上去，最終變成了十餘個 **Docker** 容器同時運行的個人伺服器。

實作內容

在 **Arch Linux** 上以 **docker compose** 容器化部署所有服務，並透過以下架構實現外部存取與安全防護：

- 反向代理：**Caddy**（含 **Cloudflare DNS Plugin**），統一管理所有子網域的 **HTTPS** 憑證自動簽發與路由
- 公網穿透：家中網路沒有公網 IP，因此使用 **Cloudflare Tunnel**（**cloudflared**）以 **host network** 模式執行，將[子網域.7279744.xyz]的流量導入本地 **Caddy**
- 內網互通：**Tailscale VPN** 建立跨裝置的私有網路，搭配 **AdGuard Home** 的 **Split DNS** 將內部請求直接指向 **Tailscale IP**，避免繞行公網
- 安全防護：**iptables** 手動設定 **INPUT DROP** 策略，僅開放 **Tailscale** 與必要端口；**CrowdSec IPS** 分析 **Caddy access log** 自動封鎖異常 IP
- 遠端 **SSH**：透過 **Cloudflare Access Zero Trust** 提供瀏覽器 **SSH**（**ssh.7279744.xyz**）並且另外架設 **GitHub OAuth** 驗證

目前執行的服務

AdGuard Home（DNS）、**Navidrome**（音樂）、**Immich**（照片）、**Dockge**（**Docker** 管理）、**ConvertX**（檔案轉換）、**2FAuth**（雙因素驗證）等。

伺服器維護與除錯實例

在維護 **Homelab** 的過程中，我的伺服器曾發生 **CPU** 無故持續滿載的異常。初期排查時我懷疑是 **Docker** 容器遭到入侵，但透過暫停 **Docker** 服務後交叉比對，發現異常源自宿主機本身。我改用 **journalctl -f** 即時監控系統日誌，最終捕捉到一個異常喚醒的 **service**，追溯後發現與我先前安裝的 **litellm** 套件有關。由於該套件透過 **Python** 的 **.pth** 機制植入了啟動腳本，一般的 **pip uninstall** 無法完全移除，我最後是手動用 **rm** 將殘留的 **.pth** 檔案與相關目錄一併刪除，系統才恢復正常。

事後我從 **The Primeagen** 的技術頻道得知，**litellm** 在同一時期遭遇了一起大規模供應鏈攻擊：攻擊者入侵了套件維護者的 **GitHub** 帳號，推送了會透過 **.pth** 機制在 **Python** 直譯器啟動時自動竊取系統憑證的惡意版本，影響數十萬名使用者。我當時觀察到的 **CPU** 滿載症狀與報導中描述的 **forkbomb** 行為完全吻合。

這次經驗讓我體認到，即使是每月下載量近億次的熱門開源套件也可能被入侵，安裝來源的安全性不能想當然。所以，系統除錯時仰賴日誌等客觀證據而非猜測，以及所有套件都在虛擬環境或容器中隔離安裝，絕不直接裝到系統全域的 **Python** 環境。這些習慣雖然不能百毒不侵，但是至少形成了最基本的防線。

反思

架設過程中最大的收穫來自於工具的替換與踩坑。一開始我照朋友的建議使用 **Nginx Proxy Manager** 的 GUI 反向代理，但在發現 **Caddy** 能自動從 **Let's Encrypt** 取得 **HTTPS** 憑證後決定轉換。不過，轉換的過程十分曲折。無論是 **Caddyfile** 的語法、**Cloudflare Tunnel** 的整合方式，還是 **redirect** 的架設方法，

每一步都跟我的直覺不同，花了大量時間反覆查閱官方文件與社群討論才逐步解決。經過這次的經驗，為了不再次犯同樣的錯誤，我建立起了一套 **SOP**。在導入新工具之前，先讀完官方文件的基礎章節再動手，而不是邊猜邊改。雖然看起來較慢，但實際上省下的除錯時間遠遠超過閱讀的時間。

